Week 8 Laboratory on Stacks

Balanced Parentheses and Arithmetic Evaluator in C++

Build a console application that uses your own stack to validate brackets, convert infix expressions to postfix, and evaluate postfix expressions. This activity follows the Week 8 practical output in the CC105 syllabus and supports CO2 and CO3.

Name or members: Serdenia, Sheayla Rihanna A.

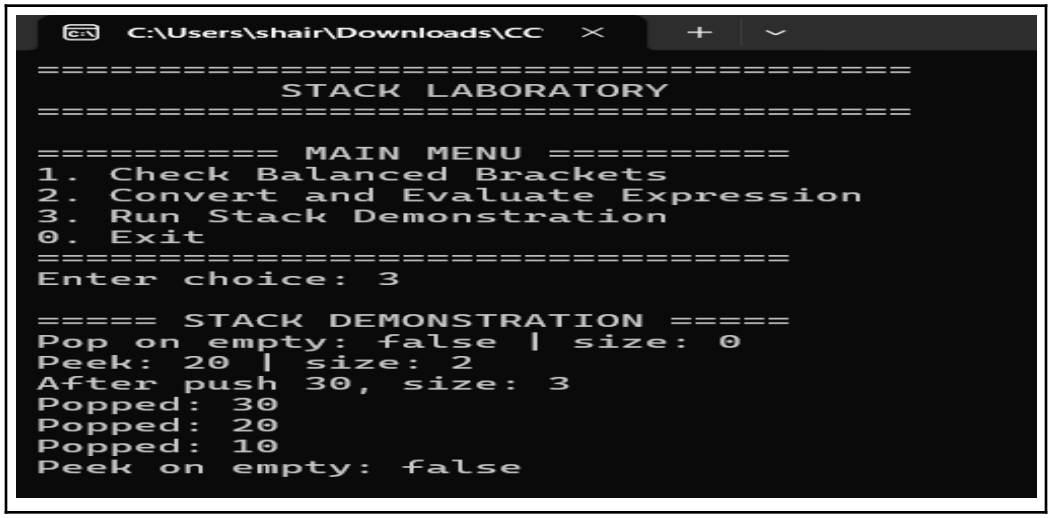
Section: 2.4BSIT Date: 11-04-2026

Repository link: https://github.com/425002643-debug/Stack-Laboratory

Part 1 Build and verify the stack

Operation	Expected outcome	Actual Output	Pass/Fail
pop on empty	false; size remains 0	false; size 0	PASS
push 10, push 20, peek	peek gives 20; size remains 2	20; size 2	PASS
push 30	array grows or third node added; size 3	Size 3	PASS
pop three times	30, then 20, then 10	30, 20, 10	PASS
peek on empty	false; no crash	false	PASS

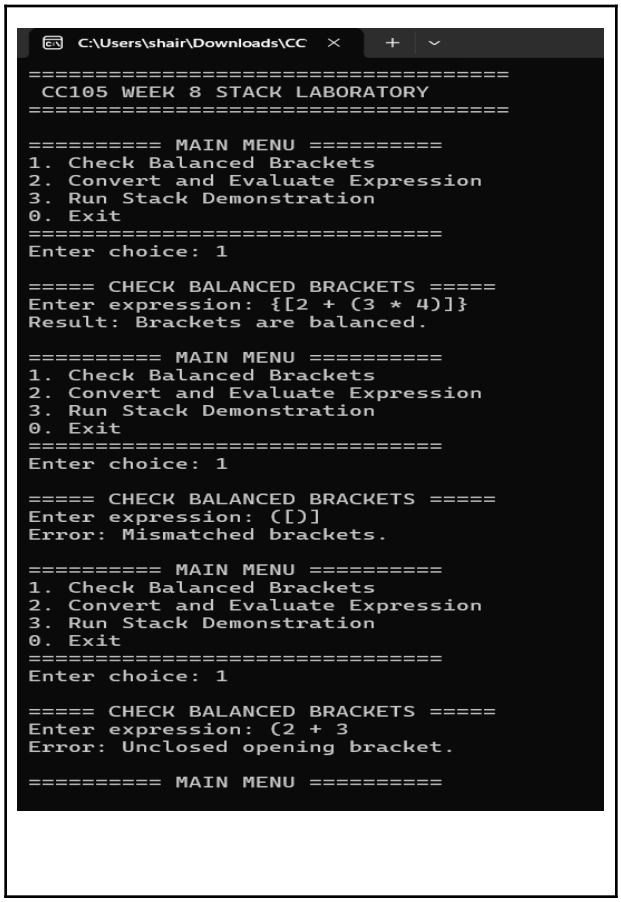
Evidence:



Part 2 Check balanced brackets

Test	Input	Actual Output	Status
Balanced brackets	{[2 + (3 * 4)]}	Brackets are balanced.	PASS
Mismatched brackets	([)]	Mismatched brackets.	PASS
Unclosed bracket	(2 + 3	Unclosed opening bracket.	PASS
Unmatched closing bracket	2 + 3)	Unmatched closing bracket.	PASS

Evidence:



Part 3 Worked conversion trace

Token	Operator stack	Postfix output
(	(	empty
12	(	12
+	(+	12
3	(+	12 3
)	empty	12 3 +
*	*	12 3 +
2	*	12 3 + 2
end	empty	12 3 + 2 *

Infix Expression: (12 + 3) * 2

Postfix Expression: 12 3 + 2 *

Final Evaluation Result: 30

Explanation: The program converts the infix expression into postfix notation using a stack. Numbers are added directly to the postfix output, while operators are temporarily stored in the stack according to the precedence. When a closing parenthesis is found.

Part 4 Evaluating Postfix

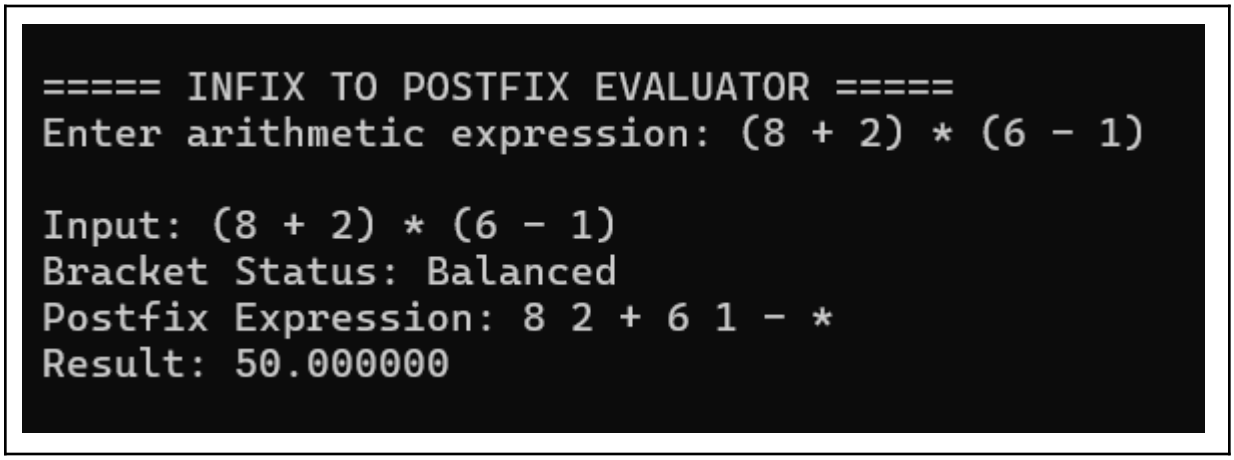
Token	Operation	Stack (bottom to top)
8	Push 8	8
2	Push 2	8, 2
+	Pop 2 and 8; calculate 8+2 = 10	10
6	Push 6	10, 6
1	Push 1	10, 6, 1
-	Pop 1 and 6; calculate 6-1 = 5	10, 5
*	Pop 5 and 10; calculate 10x5 = 50	50

Postfix Expression: 8 2 + 6 1 - *

Final Evaluation Result: 50

Explanation: The postfix expression is evaluated using a stack. Numbers are pushed onto the stack. When an operator is encountered, the right operand is popped first, followed by the left operand. The operation is performed, and the result is pushed back onto the stack.

Evidence:



Part 5 Integrate and test the application

Input	Expected result	Actual Output	Status
(12 + 3) * 2	12 3 + 2 * ; result 30	Result: 30.000000	PASS
8 + 2 * 5	8 2 5 * + ; result 18	Result: 18.000000	PASS
20 / 5 / 2	20 5 / 2 / ; result 2	Result: 2.000000	PASS
10 - 3 - 2	10 3 - 2 - ; result 5	Result: 5.000000	PASS
{[2 + 3] * (4 + 1)}	2 3 + 4 1 + * ; result 25	Result: 25.000000	PASS
7 / 2	7 2 / ; result 3.5	Result: 3.500000	PASS
0	0 ; result 0	Result: 0.000000	PASS
Invalid-expression test			
([)]	mismatched brackets	Mismatched brackets.	PASS
(2 + 3	unclosed opening bracket	Unclosed opening bracket.	PASS
2 + 3)	unmatched closing bracket	Unmatched closing bracket.	PASS
5 / (3 - 3)	division by zero	Division by zero.	PASS
2+*3	Invalid operator sequence	Invalid operator sequence or missing operand	PASS
2 3+4	Missing operator	Missing operator between operands.	PASS
2(3 + 4)	Implicit multiplication rejected	Implicit multiplication is	PASS

		not allowed.	
() or empty input	Invalid expression	Empty group or missing operand	PASS
-5 + 2	Unsupported unary sign	Invalid operator sequence or missing operand.	PASS
2.5 + 1	Unsupported decimal	Unsupported character: .	PASS
2 + a	Unsupported character	Unsupported character: a	PASS
2 +	Missing operand	Expression ends with a missing operand.	PASS

Original tests:

Input	Expected result	Actual output	Status
1+(2+(3+(4+(5+6))))	Postfix: 1 2 3 4 5 6 + + + + +; result 21	Result: 21.000000	PASS
1+2+3+4+5+6+7+8+9+10	Postfix: 1 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10; result 55	Result: 55.000000	PASS
10 + * 5	Invalid operator sequence	Invalid operator sequence or missing operand.	PASS

Add at least three original tests: a deeply nested expression, a long expression that forces array growth or many nodes, and a malformed expression. For double results, compare with a tolerance such as 0.000001 when checking automatically.

Part 6 Compare stack behavior

For this activity, I used a dynamic array to implement a stack, which follows the Last In, First Out (LIFO) principle. Unlike a linked-list implementation that stores elements in separate nodes, a dynamic array stores elements in a continuous block of memory and doubles its capacity whenever it becomes full. The push operation has an amortized time complexity of O(1), while pop and peek take O(1) time. However, resizing the array takes O(n) time because the existing elements must be copied into the new array. In comparison, a linked-list stack has O(1) time complexity for push, pop, and peek. The bracket validation, infix-to-postfix conversion, and postfix evaluation processes take O(n) time and O(n) auxiliary space because they process and store elements using a stack. Through this activity, I learned how stacks help organize data and process arithmetic expressions efficiently.

Reflection and demonstration

19. Why does LIFO correctly match nested brackets?

The LIFO or Last In, First Out helps the program to check nested parentheses because the last opening symbol that was added to the stack must be the first one to be matched by a closing symbol. This makes it easier to determine whether the parentheses, brackets and the braces are correctly arranged.

20. Why must evaluation pop the right operand before the left operand?

The right operand must be the first one to be popped because it is the last value that was pushed onto the stack. The next value popped becomes the left operand. The order is very important, especially for the subtraction and division, because changing the order can make an incorrect answer or an error. For example, in 10 2 -, the correct operation is 10 - 2, and not 2 - 10.

21. What is the difference between balanced brackets and a valid arithmetic expression?

The balance parentheses means that all of the opening and closing symbols are properly matched and neatly arranged. However, a valid arithmetic expression must also follow the correct mathematical syntax. Like for example, (2 + 3) has a balance parentheses and it is a valid expression, while (2 + *) has a balanced parentheses but it is not a valid arithmetic expression because the operators and operands are incorrectly arranged.

22. Compare your stack ADT with the function call stack. In a nested function call, which call returns first?

A Stack ADT is a data structure that I create and use in a program to store and manage data using the Last In, First Out (LIFO) principle. Meanwhile, the function call stack is managed by the system to keep track of function calls and their execution. The most recently called nested function returns first because it follows the Last-In, First-Out principle.

23. What memory is allocated and released by your implementation? Explain one resizing or node-deletion step.

The Dynamic memory allocation allows the Stack to request memory while the program is running. In my implementation, the array starts with a capacity of two elements. When the array becomes full, a new array with double the previous capacity is created. The existing elements are copied into the new array, and the old array is deleted to avoid memory leaks. When the Stack object is no longer needed, the destructor releases the remaining allocated memory using delete[].